UNIT → 4

→ Enumerators
set of named integer constants. It shows a group of constants which can't be changed

enum – enum name {
$n_1, n_2, n_3 \ldots$;
};

n_1, n_2, n_3 → names assigned to integer values
By default → $n_1 = 0, n_2 = 1, n_3 = 2$

→ Value assigned enumeerators
enum – enum name {
n_1 = value 1 , n_2 = value 2 . . . ;
};

assign n_1 then $n_2 = n_1 + 1$, $n_3 = n_2 + 1$
subsequent increment happens.

eg 1
```
#include <stdio.h>
enum direction {
EAST, NORTH, WEST, SOUTH;
};   0     1     2     3
int main () {
enum direction dir = NORTH;
```

```
printf("%d\n", dir);
return 0;
}
```

Output → 1

eg-2

```
#include <stdio.h>
enum enm {
a=3, b=2, c;
};
int main() {
enum enm v1 = a
enum enm v2 = b
enum enm v3 = c
printf("%d %d %d", v1, v2, v3);
return 0;
}
```

Output → 3, 2, 3

→ Pre processor directives
These are the instructions that are given to the compiler before actual compilation begins
They start with '#' symbols and don't end with ';'

- Use

1 Including header files
2 defining symbolic constants
3 Macros
4 conditional compilation
5 File inclusion
6 debugging

- Types

1. File inclusion (# include <file name . h>)
used to include header files like
include <stdio . h> # include <math . h>
include <conio . h>

2. Macro define ~~(# include~~ (# define _name-value)
used to define constant / expressions
# define PI 3.14
# include <stdio . h>
int main () {
printf (" Value of PI = %. f", PI);
return 0;
}

3. Macros with arguements
It works like a function
```
# include <stdio.h>
# define square (x) x*x
int main () {
printf ("square = %d", square (5));
return 0;
}
```

4. #undef
used to undefine a macro
```
# define value 10
# undef value
```

5. # pragma → diff compilers diff outputs
used to give special instructions to compiler
```
# pragma warn
# pragma startup
# pragma exit
# pragma once
```

__/__/____

→ Macro defination
In c language, macro is a pre processor feature used to define symbolic constants/ reusable expressions. It's used using # define directive & processed by pre processor before compilation.

Macros are mainly used to improve readability reduce repetition & make programs easier to modify.

A macro doesn't occupy memory, because it's simply replaced by it's value before the compilation

- Object like macros (constant macros)

```
# include <stdio.h>
# define PI 3.14
int main () {
print f ("%.f", PI);
 return 0;
  }
```

- Function like macros

```
# include <stdio.h>
# define square (x)  x*x
int main() {
print f ("%.d", square (5));
 return 0;
 }
```

→ conditional compilation

It means compiling certain parts of program only when a specific condition is true. It allows compiler to include / exclude the parts of source code depending on the conditions defined

- #if

used to test a constant / integer condition

```
#include <stdio.h>
#define value 10
int main() {
#if value > 5
 printf ("value is greater than 5");
#endif
  return 0;
  }
```

- #else

Determines alternative cond when if fails

```
#include <stdio.h>
#define num 3
int main() {
#if num > 5
printf ("Greater than 5");
#else
 printf ("Not greater than 5");
#endif
 return 0;
  }
```

__/__/____

- # elif (else if)
used to check another condition if first is false

```
# include <stdio.h>
# define x 0
int main () {
# if x > 0
  printf ("Positive");
# elif x < 0
  printf ("Negative");
# else
  printf ("Zero");
# end if
  return 0;
  }
```

- # ifdef
True if macro is already defined

```
# include <stdio.h>
# define INDIA
int main () {
# ifdef INDIA
 printf ("Welcome to INDIA");
# end if
 return 0;
 }
```

- # ifndef
True if macro isn't defined

```
# include <stdio.h>
# ifndef count
# define count 10
# end if
int main () {
printf ("%d", count);
return 0;
}
```

- # end if
Marks the end of conditional block (compulsory closing)
Used with every # if, # else, # elif.

__/__/____

→ Storage classes

In C language, a storage class defines 4 important properties properties of a variable

1. Lifetime
   ~ How long a variable exists in memory
2. Scope
   ~ where in the program its accessible
3. Default initial value
4. Storage location
   ~ memory / CPU register

Storage class tells the compiler how to store, and how long to store a variable.

- Auto storage class
- Declares automatic / local variables
- created when func starts & destroyed when func ends.
- Syntax → auto — int — x ;

Scope	local to block / func
Lifetime	only during func execution
Default value	garbage
Storage	RAM

```
void func ()
{
  auto int x = 10;
  printf ("%d", x);
}
```

→ Register storage class

- Stores variable in CPU register for faster access
- Syntax → register int i;

Scope	Local
Lifetime	Func execution only
Default value	Garbage
Stored in	CPU registers

```
void fun () {
register int i;
  for (i=0; i<=100; i++) {
  printf ("%d", i);
    }
  }
```

→ Static storage class

- Retains value even after func ends
- Allocated only once
- Syntax → static int x;

Scope	Local / Global
Lifetime	Entire Program
Default value	0
Stored in	RAM, static area

```
void counter () {
static int x = 0;
  x++;
printf ("%d", x);
  }
```

→ extern storage class
- used to declare global variable in another file
- doesn't allocate memory, only refers to existing variable
- syntax → extern int x;

Scope	Global
Lifetime	Entire program
Default value	0
Stored in	RAM

→ Dynamic Memory Allocation
It refers to allocating memory at runtime instead of compile time. It requests memory when needed, release memory when not needed & use memory efficiently
eg → #include <stdlib.h>

malloc ()	Allocates memory block
calloc ()	Allocate memory & initialise to 0
realloc ()	change/extend prev. allocated memory
free ()	Releases memory

1. malloc ()
- Syntax pointer = (datatype*) malloc (size-in-bytes);
- Allocates memory of required size
- Returns void* pointer
- Initial value → Garbage

```
int *p;
p = (int *) malloc (5 * sizeof (int));
```

2. calloc ()
- ptr = (type *) calloc (n, size);
- allocates continous size of memory
- initializes all elements to 0.

eg :-
```
int *p;
p = (int *) calloc (5, size of (int));
```

3. realloc ()
- used to resize memory allocated by malloc/ calloc
- ptr = realloc (ptr, new – size);

eg :-
- p = realloc (p, 10 * size of (int));
  ↳ extends size of array from 5 to 10

4. free ()
- used to release allocated memory
- syntax → free (ptr);
- Prevents memory leak & wastage of RAM

→ eg :-
```
# include <stdio.h>
# include <stdlib.h>
int main () {
int n, *p;
printf ("enter size :");
scanf ("%d", &n);
p = (int *) malloc (n * size of (int));
 for (int i = 0, i<n, i++) {
  scanf ("%d", &p[i]);
}
```

```
printf("Elements : ");
for(int i=0; i<n; i++){
  printf("%d", p[i]);
free(p);
  return 0;
  }
```

→ File Handling

It means performing operations on files stored in secondary memory. Using files, data becomes permanent even after program termination.

- File pointer

File in C handled using file pointer

```
FILE * fp;
```

fp is used to access a file

- opening a file

```
fp = fopen("filename", "mode");
```

↳ To open a file we use

eg :-

```
FILE * fp
fp = fopen("data.txt", "r");
```

- File's modes

r	open to read that file
w	open for writing, (creates new file, deletes old)
a	open for appending, (add data at end)
r+	read + write
w+	write + read
a+	append + read

- closing a file
  After operations, files must be closed
  fclose (fp);

- File input / output functions
  → character based
    fgetc () reads a character
    fputc () writes a char.

  eg → fp = fopen ("abc.txt", "w");
    fputc ('A', fp);
    fclose (fp);

  → string based
    fgets () reads a string
    fputs () writes a string

  → Formatted input/output
    fscanf () formatted reading
    fprintf () formatted writing

  → eg to write a file

```
#include <stdio.h>
int main () {
  FILE * fp;
  fp = fopen ("data.txt", "w");
fprintf (fp, "Hello file handling");
  fclose (fp);
  return 0;
  }
```

→ eg to read a file

```
#include <stdio.h>
int main () {
FILE * fp ;
char ch;
fp = fopen ("data.txt", ("r"));
while ((ch = fgetc(fp)) != EOF)
{ printf ("%c", ch);
  }
fclose (fp);
 return 0;
    }
```

→ Text file
It stores data in human readable form

→ Binary file
It stores data in binary form.